

编译原理

7. 中间代码生成

rainoftime.github.io

**浙江大学
计算机科学与技术学院**

Content

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
- 7. Translating into Intermediate Code**
8. Basic Blocks and Traces
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

本讲内容

1

中间表示概述

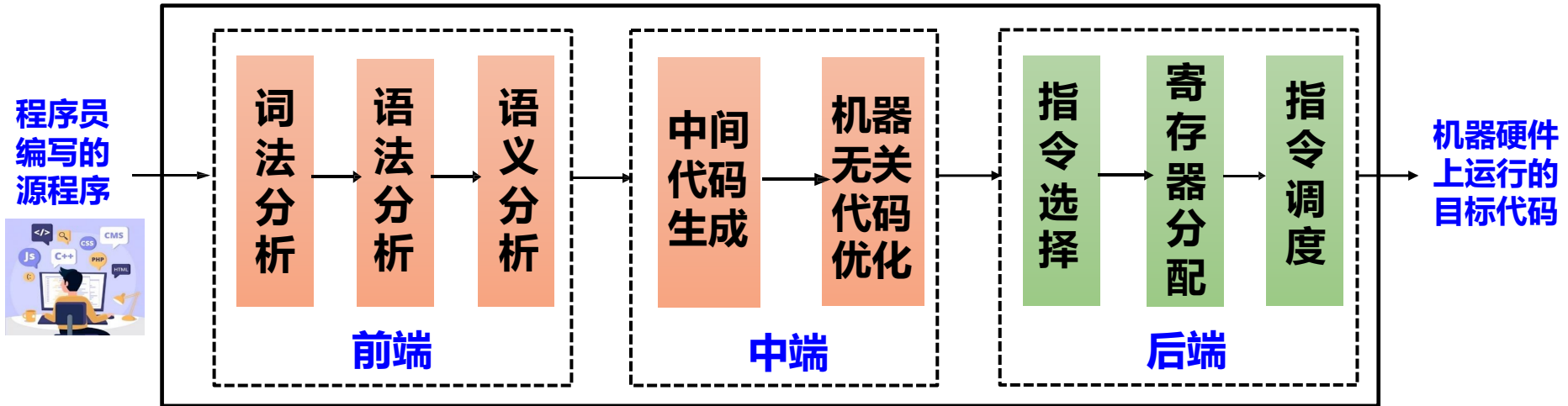
2

IR Tree中间表示

3

IR Tree的生成

基本概念回顾



前端 – 从源码到IR生成

中端 – 基于IR的分析与变换（可能生成新IR）

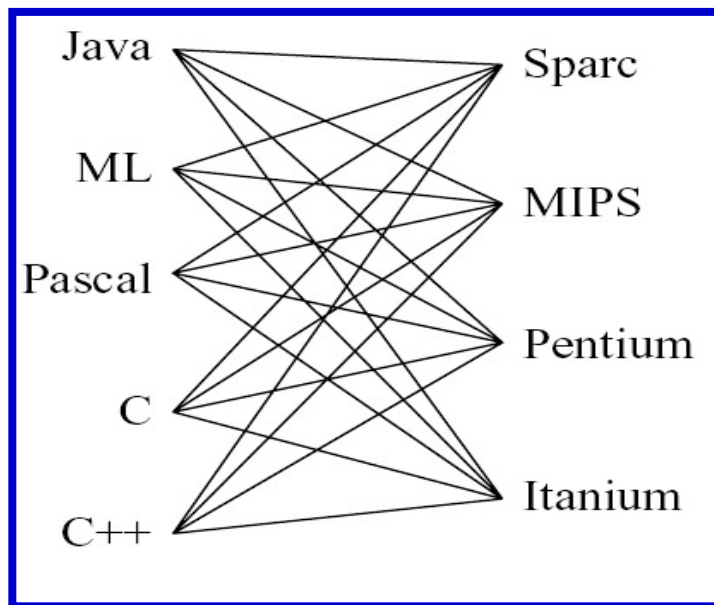
后端 – (机器相关优化)；从IR 到目标代码

1. 中间表示概述

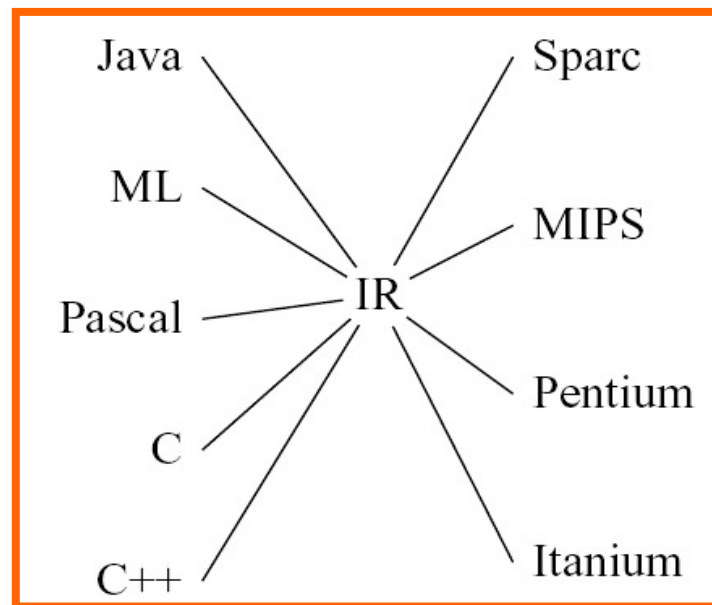
- **为什么需要中间表示(IR)**
- **(不要求掌握)IR分类: 根据抽象层次**
- **(不要求掌握)IR分类: 根据结构特征**
- **三地址码**

为什么需要中间语言/表示

- How about translating directly to real machine code (e.g., via semantic actions?)
 - hinders **modularity**
 - hinders **portability**



$N * M$



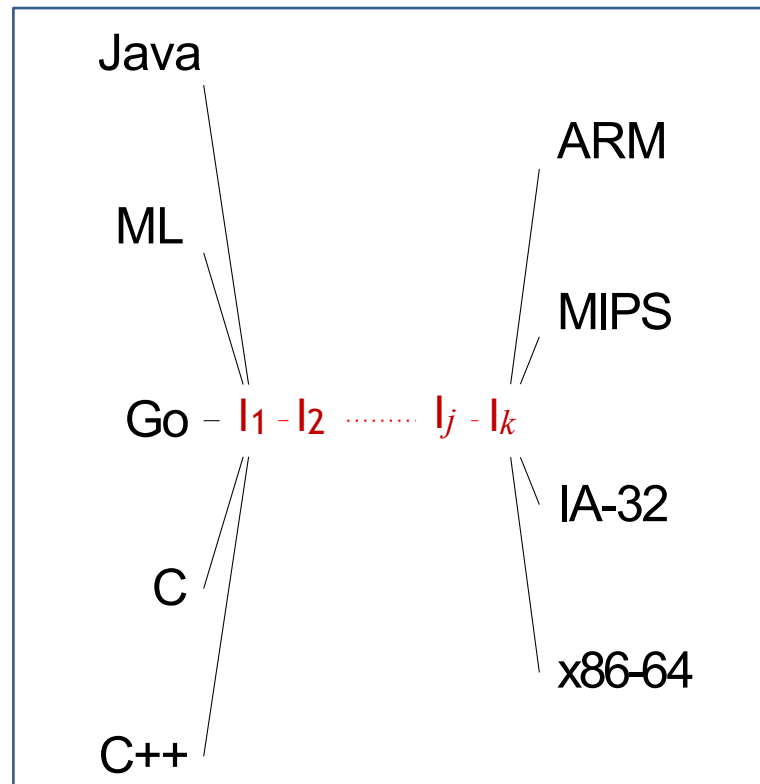
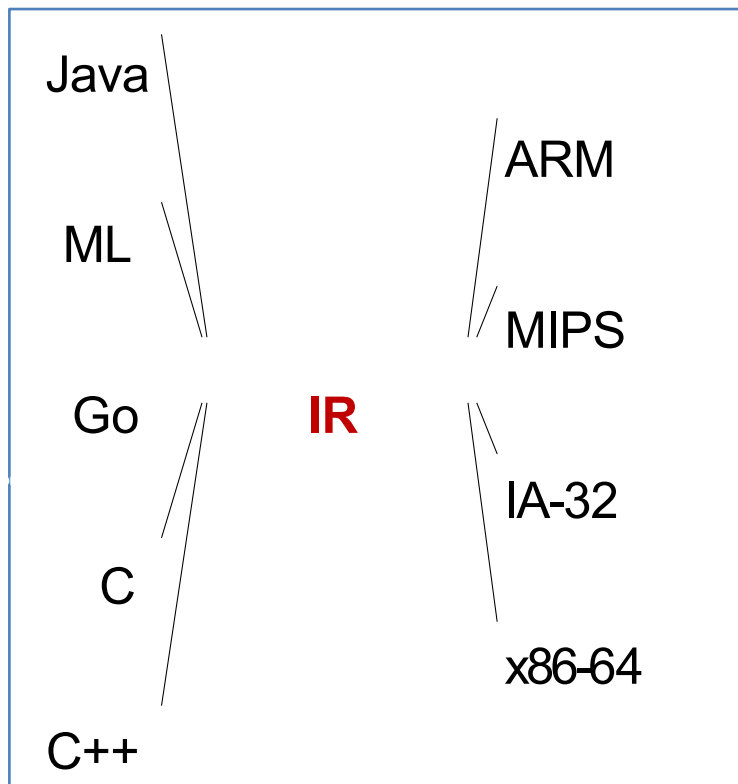
$N + M$

1. 中间表示概述

- 为什么需要中间表示(IR)
- (不要求掌握)IR分类: 根据抽象层次
- (不要求掌握)IR分类: 根据结构特征
- 三地址码

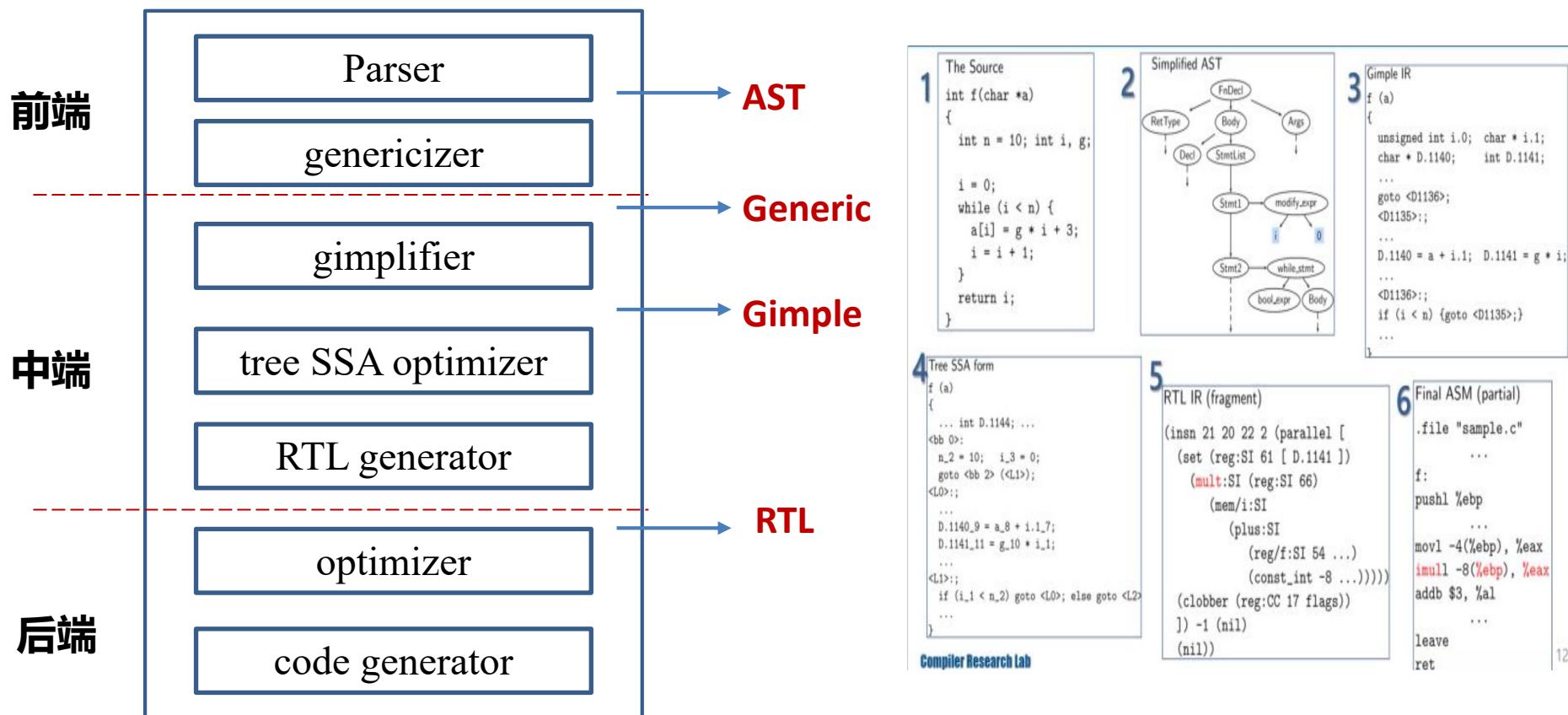
中间表示分类: 抽象层次

- 实际编译器可能采用**多层IR**
 - 支持不同层次的**分析和变换**



中间表示分类: 抽象层次

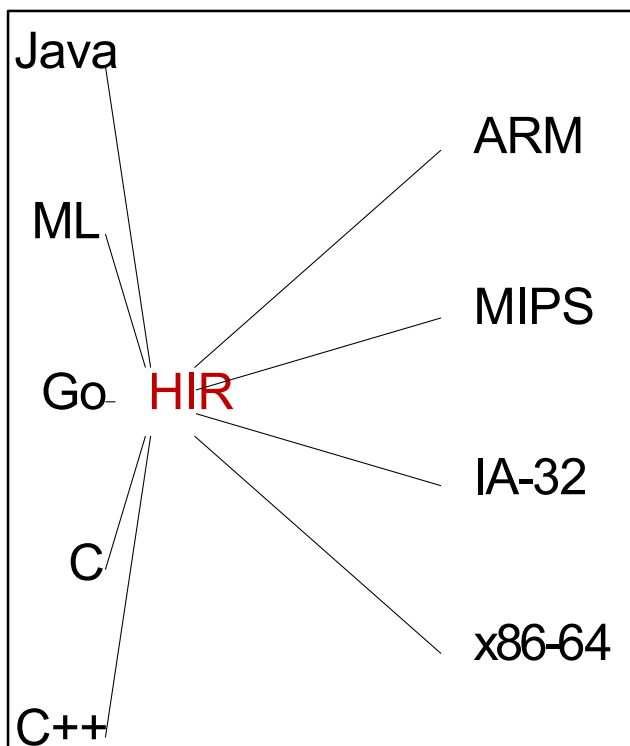
- 实际编译器可能采用**多层IR**
 - 支持不同层次的**分析和变换**



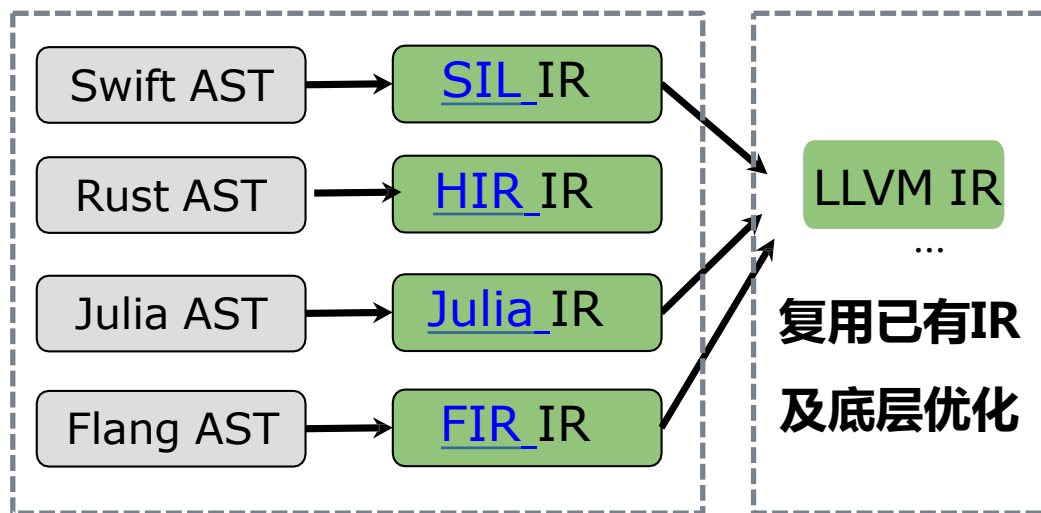
GCC编译器的多层IR

中间表示分类: 抽象层次

- **高层中间表示 High-level IR**
 - 贴近输入语言，方便由前端生成
- **低层中间表示 Low-level IR**
- **中层中间表示 Middle-Level IR**



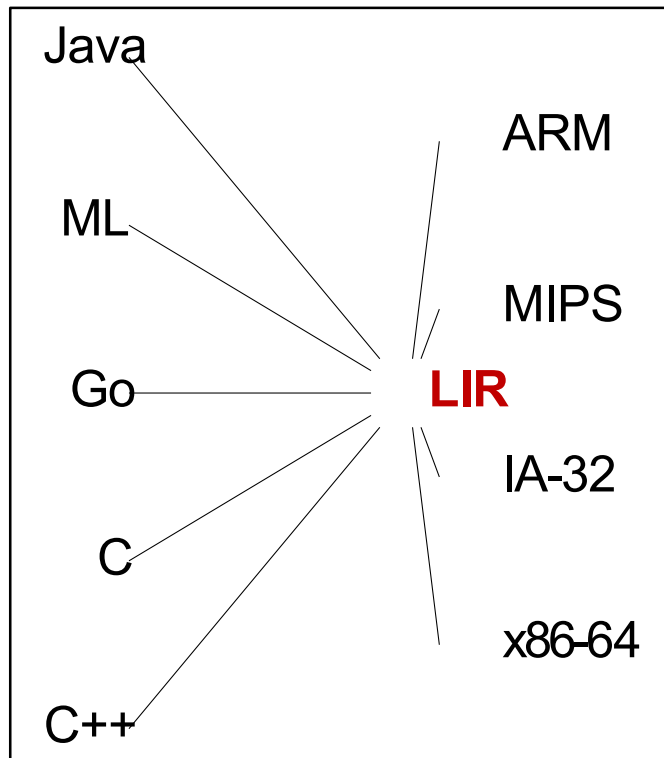
现代编程语言不断推出高级中间表示



“高级” 类型检查(如borrow checker)、
从高层抽象逐步 "lowering"

中间表示分类: 抽象层次

- 高层中间表示 High-level IR
- **低层中间表示 Low-level IR**
 - 贴近目标语言，方便目标代码生成
- 中层中间表示 Middle-Level IR



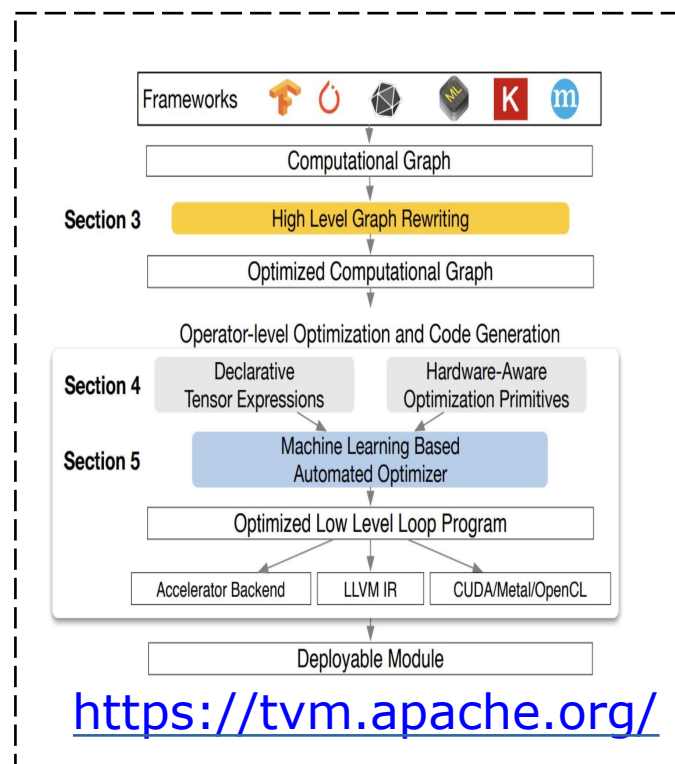
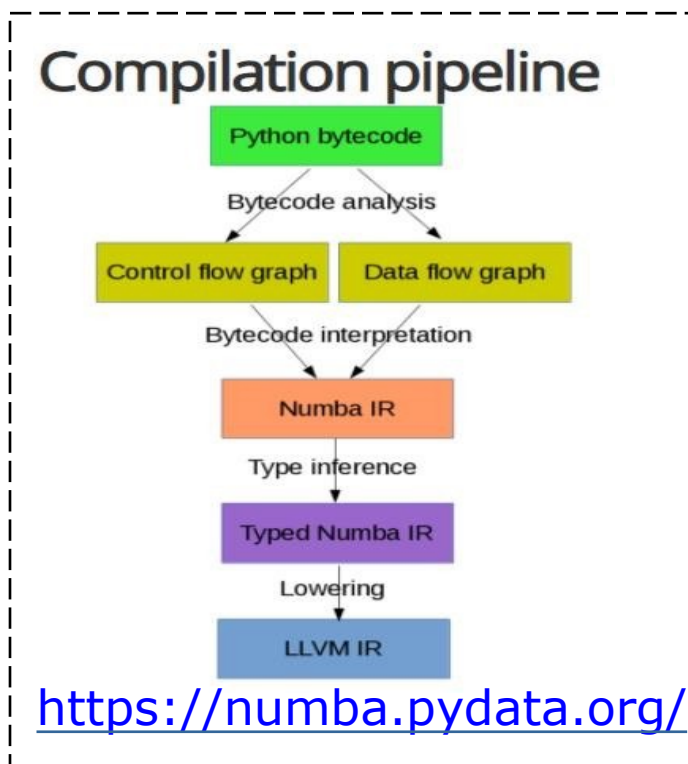
GCC编译器的RTL中间表示

```
(note 45 44 46 ("gcd.c") 11)
(note 46 45 47 NOTE_INSN_DELETED)
(note 47 46 49 NOTE_INSN_DELETED)
(insn 49 47 51 (set (reg:SI 64)
  (mem/f:SI (reg/f:SI 53 virtual-incoming-args) [0 a+0 S4 A32]))
  (nil))
(insn 51 49 52 (set (reg:SI 58)
  (reg:SI 64)) -1 (nil)
  (nil))
(jump_insn 52 51 53 (set (pc)
  (label_ref 56)) -1 (nil)
  (nil))
(barrier 53 52 54)
(note 54 53 55 NOTE_INSN_FUNCTION_END)
(note 55 54 59 ("gcd.c") 12)
(insn 59 55 60 (clobber (reg/i:SI 0 eax)) -1 (nil)
  (nil))
(insn 60 59 56 (clobber (reg:SI 58)) -1 (nil)
  (nil))
```

中间表示分类: 抽象层次

- 高层中间表示 High-level IR
- 低层中间表示 Low-level IR
- 中层中间表示 Middle-Level IR

注意：确切地说，三者之间没有严格的界限

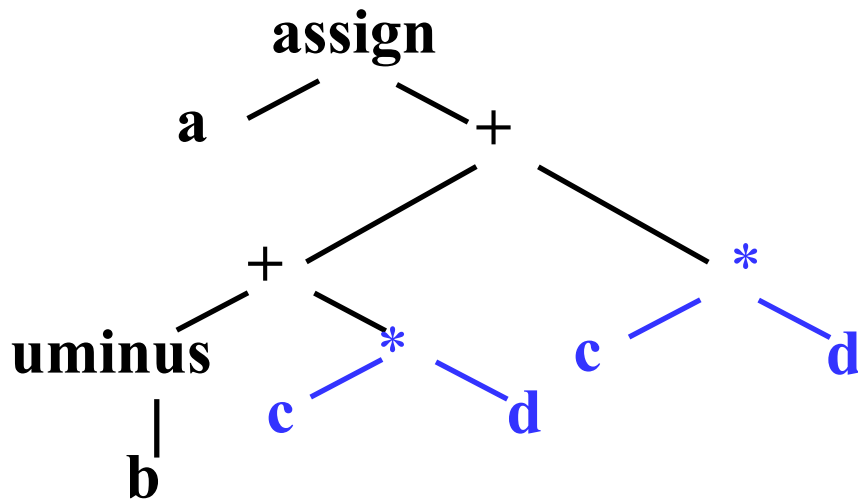


1. 中间表示概述

- 为什么需要中间表示(IR)
- (不要求掌握)IR分类: 根据抽象层次
- (不要求掌握)IR分类: 根据结构特征
- 三地址码

中间表示分类: 结构特征

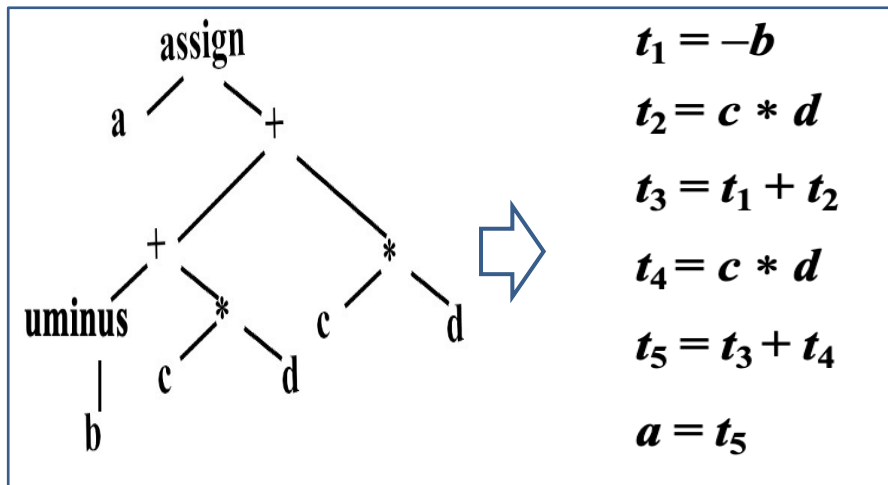
- **结构化表示 Structural**
 - Graphically oriented (e.g., tree, DAG,...)
 - Heavily used in source-to-source translators
- **线性表示 Linear**
- **混合表示 Hybrid**



$$a = (-b + c*d) + c*d$$

中间表示分类: 结构特征

- 结构化表示 Structural
- 线性表示 Linear
 - 线性表示: 存储布局是线性的
- 混合表示 Hybrid



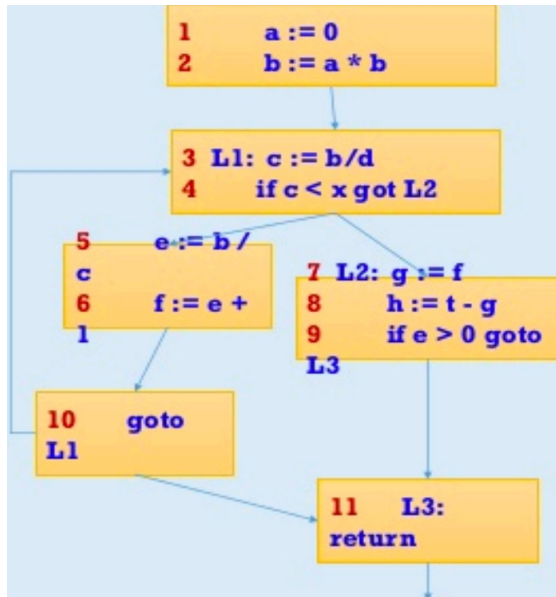
(a) 三地址码 three address code

```
% gcj-3.3 -c gcd.java
% jcf-dump-3.3 -c gcd
...
Method name:"gcd" public static
Signature: 5=(int,int)int Attribute
"Code", length:66, max_stack:2,
max_locals:2, code_length:26
  0: iload_0
  1: iload_1
  2: if_icmpeq 24
  5: iload_0
  6: iload_1
  7: if_icmple 17
```

(b) stack machine code

中间表示分类: 结构特征

- 结构化表示 Structural
- 线性表示 Linear
- 混合表示 Hybrid
 - Combination of graphs and linear code



- 节点内的语句是线性表示
- 节点间构成图形化表示

例: 控制流图 Control-flow graph

1. 中间表示概述

- 为什么需要中间表示(IR)
- (不要求掌握)IR分类: 根据抽象层次
- (不要求掌握)IR分类: 根据结构特征
- **三地址码 (Three-Address Code)**

三地址码 (Three-Address Code)

- **目标**

- 接近大多数目标机器的执行模型 (机器码)
- 支持大多数目标机器提供的数据类型和操作
- 提供有限度的、高于机器码的抽象表达能力

- **三地址码的一般形式： $x = y \text{ op } z$**

- 每个指令最多1个算符，最多3个操作数(三地址)
- 例: 表达式 $x + y * z$ 翻译成的三地址语句序列是

三地址码 (Three-Address Code)

- 例: 表达式 $x + y * z$ 翻译成的三地址语句序列是
 - $t_1 = y * z$ $t_2 = x + t_1$

序号	指令类型	指令形式
1	赋值指令	$x = y$ op z $x =$ op y
2	复制指令	$x = y$
3	条件跳转	if x relop y goto n
4	非条件跳转	goto n
5	参数传递	param x
6	过程调用	call p, n
7	过程返回	return x
...	...	...

“地址”可以具有如下形式

- 源程序中的**名字** (*name*)
- **常量** (*constant*)
- **临时变量** (*temporary*)
- ..?

例: 三地址码

High-level language

```
read x ; { input an integer }  
if 0 < x then { don't compute if x <= 0 }  
    fact:=1;  
repeat  
    fact:=fact*x;  
    x:=x-1  
until x=0;  
write fact { output factorial of x }  
end
```

Three-address code

```
read x  
t1=x>0  
if_false t1 goto L1  
fact=1  
label L2  
t2 = fact * x  
fact = t2  
t3 = x - 1  
x = t3  
t4= x==0  
if_false t4 goto L2  
write fact  
label L1  
halt
```

Three-Address Code - Implementation

- The entire sequence of three-address instructions is implemented as an array of linked list
- The most common implementation is to implement three-address code as **quadruples**:
 - one field for the operation
 - three fields for the addresses
- For those instructions that need fewer than three addresses, one or more of the address fields is given a null or “empty” value.

t1=x>0	(gt, x, 0, t1)
if_false t1 goto L1	(if_f, t1, L1, _)
fact=1	(asn, 1, fact, _)
label L2	(lab, L2, _, _)
- Other implementation: **triples**, **indirect triples**.

静态单赋值 (SSA) (实验可能需要, 但不考)

- Static Single Assignment (SSA) 是一种特殊的三地址代码, 其所有变量在代码中只被赋值一次

$x = a;$

$y = x + 1;$

$x = b;$

$z = x * 2;$

$x_1 = a;$

$y = x_1 + 1;$

$x_2 = b;$

$z = x_2 * 2;$

- 方便了编译器中的很多分析和优化
 - 方便查询信息(比如def-use, dependence)
 - 加速现有算法(基于SSA的稀疏分析)
 - 严格依赖SSA的算法(ssapre, new gvn,...)
 - “免费”提升精度 (如获得部分流敏感效果)

广泛使用
于现代
编译器中
(如LLVM)

2. Intermediate Representation Trees

Intermediate Representations in Tiger

- Some modern compilers use several IRs -- each IR in later phase is a little closer (to the machine code) than the previous phase

AST ==> IR1 ==> IR2 ... ==> IRk ==> machine code

- The Tiger compiler uses one IR only --- the Intermediate Representation (IR) Tree

AST => IR Tree => assembly => machine code

IR Tree: stay in the middle of AST and assembly!

IR Tree: A Low Level Tree Representation

- 一种特殊的树型中间语言/中间表示

BNF形式的文法描述

```
<Exp> ::= "CONST" int  
        | "NAME" <Label>  
        | "TEMP" <Temp>  
        | "BINOP" <Oper> <Exp> <Exp>  
        | "MEM" <Exp>  
        | "CALL" <Exp> [ {<Exp>} ] "call end"  
        | "ESEQ" <Stm> <Exp>  
  
<Stm> ::= "MOVE" <Exp> <Exp>  
        | "EXP" <Exp>  
        | "JUMP" <Exp> [ {<Label>} ]  
        | "CJUMP" <Relop> <Exp> <Exp> <Label> <Label>  
        | "SEQ" [ {<Stm>} ] "seq end"  
        | "LABEL" <Label>  
  
<Oper> ::= "ADD" | "SUB" | "MUL" | "DIV" | "MOD"  
<Relop> ::= "EQ" | "NE" | "LT" | "GT" | "LE" | "GE"
```

IR Tree: The Expressions

The computation of some value, possibly with side effects

CONST(i)	The integer constant i
NAME(n)	The symbolic constant n (e.g. label) 比如goto .L中这个.L就是NAME
TEMP(t)	Temporary t. virtual register, 不考虑寄存器个数
BINOP(o, e1, e2)	The application of binary operator o to operands e1, e2. The integer arithmetic operators are PLUS , MINUS , MUL , DIV ; the integer bitwise logical operators are AND , OR , XOR ; the integer logical shift operators are LSHIFT , RSHIFT ; the integer arithmetic right-shift is ARSHIFT .
MEM(e)	The contents of wordSize bytes of memory starting at address e. When MEM is used as the left child of a MOVE, it means “store”, but anywhere else it means “fetch”. 比如x86里的[rax] = 100, rbx = [rax]
CALL(f, l)	A procedure call: the application of function f to argument list l.
ESEQ(s, e)	Statement s is evaluated for side effects, then e is evaluated for a result. 先计算 stm s,再根据 stm s 计算 exp e,得出结果

Example: Expressions

Node	Description	Example
CONST(i)	Integer constant i	CONST(42) $\rightarrow$ the value 42
NAME(n)	Symbolic constant (label)	NAME(L1) $\rightarrow$ address of label L1
TEMP(t)	Temporary (like register)	TEMP(t123) $\rightarrow$ contents of temporary t123
BINOP(o,e1,e2)	Binary operation	BINOP(PLUS,TEMP(t1),CONST(1)) $\rightarrow$ t1+1
MEM(e)	Memory access	MEM(CONST(100)) $\rightarrow$ contents at address 100
CALL(f,l)	Function call	CALL(NAME(print),[TEMP(t1)]) $\rightarrow$ call print(t1)
ESEQ(s,e)	Statement then expression	ESEQ(MOVE(TEMP(t),CONST(1)),TEMP(t)) $\rightarrow$ (t=1; t)

IR Tree: The Statements

Perform side-effects and control flow - no return value!

MOVE(TEMP t, e)	Evaluate e and move it into temporary t .
MOVE(MEM(e₁) e₂)	Evaluate e₁ , yielding address a . Then evaluate e₂ , and store the result into wordSize bytes of memory starting at a . 计算 e1 ,得到地址 a ,再计算 e2 ,放入地址 a
EXP(e)	Evaluate e and discard the result.
JUMP(e, labs)	Transfer control (jump) to address e . The destination e may be a literal label, as in NAME(lab) , or it may be an address calculated by any other kind of expression.
CJUMP(o, e₁, e₂, t, f)	Evaluate e₁ , e₂ in that order, yielding values a , b . Then compare a , b using the relational operator o . If the result is true , jump to t ; otherwise jump to f .
SEQ(s₁, s₂)	The statement s₁ followed by s₂ .
LABEL(n)	Define the constant value of name n to be the current machine code address.

注意Label和Expression中的Name的区别。Name是使用这个symbol，比如goto .L，而Label是定义这个symbol，即.L:

Example: Statements

Node	Description	Example
MOVE(TEMP t, e)	Assign to a temporary	MOVE(TEMP(t1),CONST(42)) → t1 = 42
MOVE(MEM(e1), e2)	Store to memory	MOVE(MEM(TEMP(t1)), CONST(42)) → *t1 = 42
EXP(e)	Evaluate and discard	EXP(CALL(NAME(print), ...)) → call print() for effect
JUMP(e, labs)	Jump to address	JUMP(NAME(L1), [L1]) → goto L1
CJUMP(o,e1,e2,t,f)	Conditional jump	CJUMP(LT,TEMP(t1),CONST(0),L1,L2) → if t1<0 goto L1 else goto L2
SEQ(s1, s2)	Statement sequence	SEQ(MOVE(...), JUMP(...)) → assign then jump
LABEL(n)	Define a label	LABEL(L1) → L1:

Discussion: 关于ESEQ的理解

- **ESEQ(s, e)** : The statement **s** is evaluated for side effects, then **e** is evaluated for a result.
 - 假设**s**是statement $a=5$, **e**是expression $a+5$
 - Statement (如 $a=5$)不返回值,但是有副作用
 - 表达式ESEQ($a=5$, $a + 5$)最终的结果是10
- 关于副作用(Side effects) (**重要**)
 - Side-effects means **updating** the contents of a **memory cell** or a **temporary register**

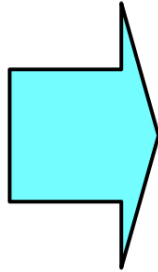
Example: IR Tree VS. X86-64 Instructions

Intel syntax	IR equivalent
17	CONST(17)
rax	TEMP(rax)
[rax]	MEM(TEMP(rax))
[rbx + 32]	MEM(ADD(TEMP(rax), CONST(32)))
[rax + rbx*8]	MEM(ADD(TEMP(rax), MUL(CONST(4), TEMP(rbx))))

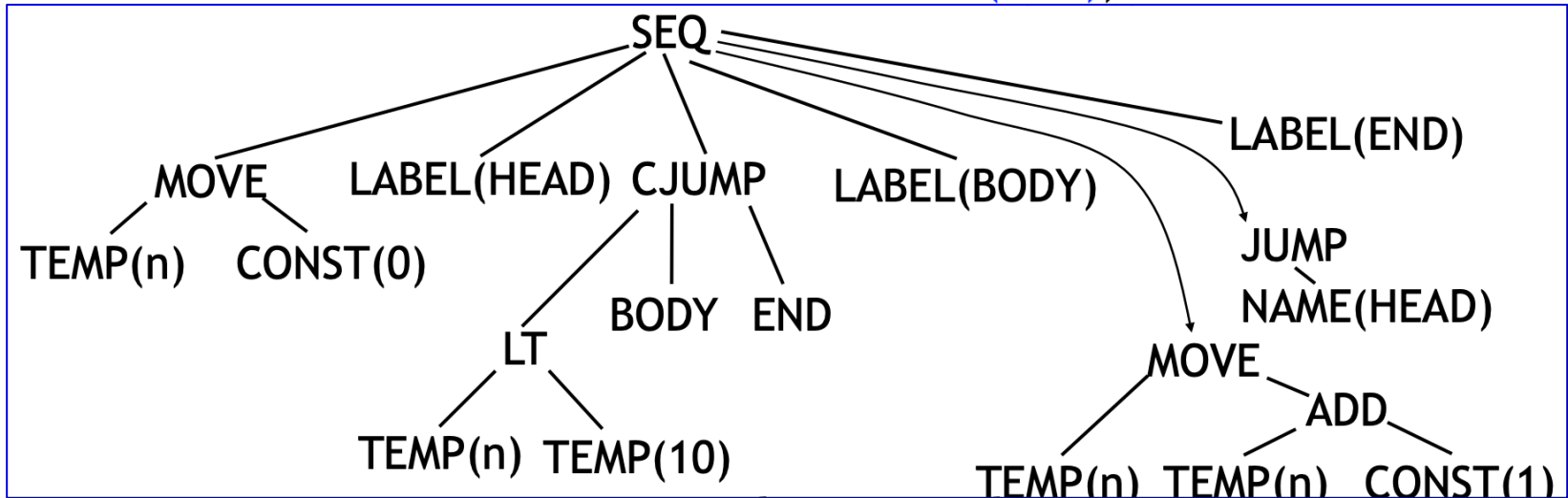
- 注: 以上并非“IR Tree翻译到机器指令的规则”

Example: IR Tree

```
n = 0;  
while (n < 10) {  
    n = n + 1  
}
```



```
SEQ(MOVE(TEMP(n), CONST(0)),  
    LABEL(HEAD),  
    CJUMP(LT(TEMP(n), CONST(10)),  
          BODY, END),  
    LABEL(BODY),  
    MOVE(TEMP(n), ADD(TEMP(n),  
                      CONST(1))),  
    JUMP(NAME(HEAD)),  
    LABEL(END))
```



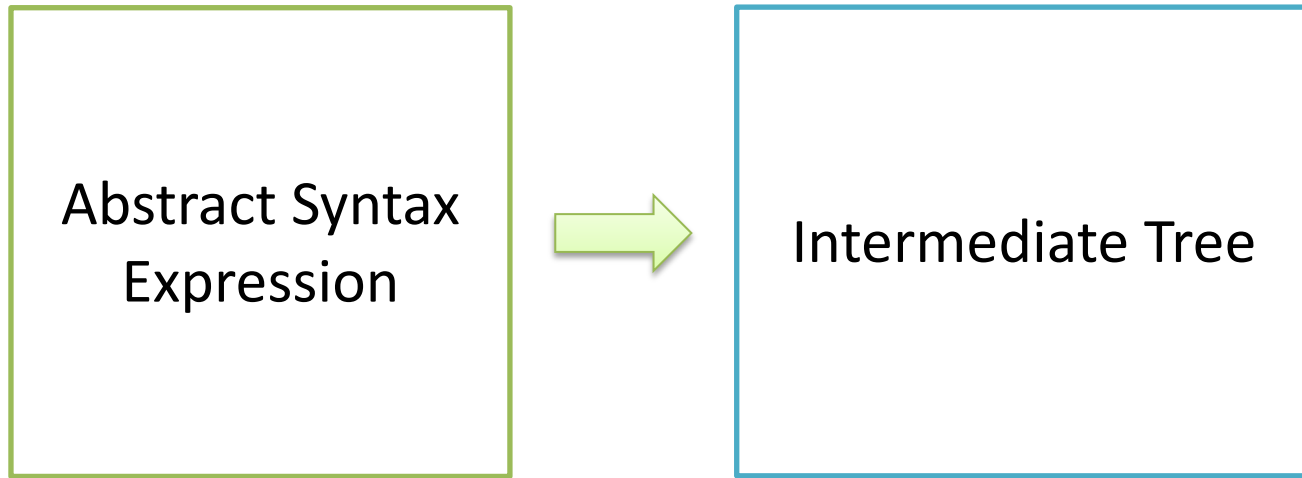
How to generate the above trees?

3. 翻译到IR Trees

Outline of the Translation

- **Translation of expressions**
 - **Overview**
 - Simple Variables
 - Array Variables
 - Structured L-values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- **Translation of declarations**
 - Variable Definition
 - Function Definition

Translating Tiger AST to IR Trees



- Tree representation is also used in compilers
- Example: the RTL IR in GCC

Review: The Tiger Language

- A tiger program consists of
 - Expressions
 - Declarations
 - Variable
 - Function
 - Type

```
exp = VarExp of var
      | NilExp
      | IntExp of int
      | StringExp of string * pos
      | AppExp of {func: Symbol.symbol, args: exp list, pos: pos}
      | OpExp of {left: exp, oper: oper, right: exp, pos: pos}
      | RecordExp of {typ: Symbol.symbol, pos: pos,
                       fields: (Symbol.symbol * exp * pos) list}
      | SeqExp of (exp * pos) list
      | AssignExp of {var: var, exp: exp, pos: pos}
      | IfExp of {test: exp, then': exp, else': exp option, pos: pos}
      | WhileExp of {test: exp, body: exp, pos: pos}
      | ForExp of {var: Symbol.symbol, lo: exp, hi: exp,
                    body: exp, pos: pos}
      | BreakExp of pos
      | LetExp of {decs: dec list, body: exp, pos: pos}
      | ArrayExp of {typ: Symbol.symbol, size: exp,
                       init: exp, pos: pos}

dec = VarDec of {var: Symbol.symbol, init: exp, pos : pos,
                 typ: (Symbol.symbol * pos) option}
      | FunctionDec of fundec list
      | TypeDec of {name: Symbol.symbol, ty: ty, pos: pos} list
```

用Standard ML语言定义的Tiger抽象语法

Mapping AST Expressions to IR Tree

- What should A_exp become in the Tree language?
 - **Ex**: Expressions with **return values** (e.g., $a + b$)
 - **Nx**: Expressions that **return no value** (e.g., `print(x)`)
 - **Cx**: Expressions with **Boolean values (conditional jump)**

AST	(Mapped) IR Tree
Ex	Tree Expressions
Nx	Tree Statement
Cx	Tree Expressions that determine..

Different language constructs naturally fit different representations

Examples

Ex

Expressions that produce a value

Examples:

- `a + b` → `BINOP(PLUS, TEMP(a), TEMP(b))`
- `f(x, y)` → `CALL(NAME(f), [TEMP(x), TEMP(y)])`
- `a[i]` → `MEM(+ (TEMP(a), * (TEMP(i), CONST(4))))`

Nx

Statements with side effects but no value

Examples:

- `a := b` → `MOVE(TEMP(a), TEMP(b))`
- `print(x)` → `EXP(CALL(NAME(print), [TEMP(x)]))`
- `while x>0 do ...` → sequence of jumps and labels

Cx - Conditional Expression

Expressions that determine control flow with true/false branches

Examples:

- `a < b` → `CJUMP(LT, TEMP(a), TEMP(b), t, f)` where t and f are labels to be filled in later
- `a & b` → short-circuit evaluation that jumps to true or false label

Conversions between the Representations

- **The Problem:** language constructs often need to be used in contexts that expect a different form

Need $E_x \rightarrow C_x$ conversion:

```
if (x+1) then ... else ..
```

$x+1$ produces a value (E_x), but the if statement needs a condition (C_x)

Need $E_x \rightarrow N_x$ conversion:

```
add_value() (called for side effects only)
```

Function returns a value (E_x), but we discard it (N_x)

Need $C_x \rightarrow E_x$ conversion:

```
y := (x > 0)
```

$x > 0$ is a condition (C_x), but assignment needs a value (E_x)

Need $C_x \rightarrow N_x$ conversion:

```
if x>0 then print("positive");
```

The entire if statement is N_x (no result), but contains a condition (C_x)

Conversions between the Representations

- **Solution:** use **utilities** for conversion among **Ex**, **Nx**, and **Cx**

Need Ex $\rightarrow$ Cx conversion:

```
if (x+1) then ... else ..
```

$x+1$ produces a value (Ex), but the if statement needs a condition (Cx)

Need Ex $\rightarrow$ Nx conversion:

```
add_value() (called for side effects only)
```

Function returns a value (Ex), but we discard it (Nx)

Need Cx $\rightarrow$ Ex conversion:

```
y := (x > 0)
```

$x > 0$ is a condition (Cx), but assignment needs a value (Ex)

Need Cx $\rightarrow$ Nx conversion:

```
if x>0 then print("positive");
```

The entire if statement is Nx (no result), but contains a condition (Cx)

Without these conversions, we would need separate translation rules for each context a construct appears in, which would be complex and error-prone.

Outline of the Translation

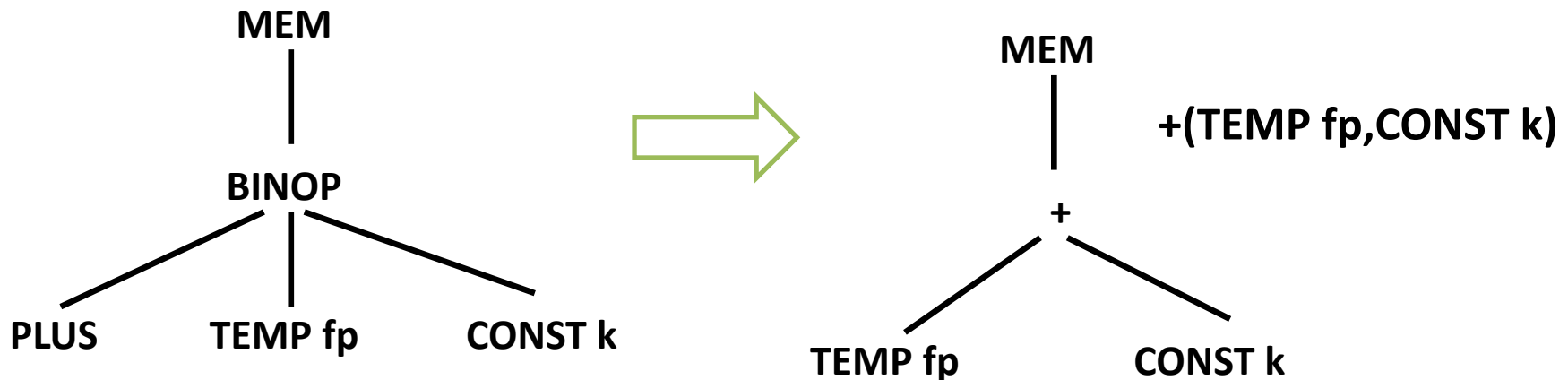
- **Translation of expressions**
 - Overview
 - **Simple Variables**
 - Array Variables
 - Structured L-values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- **Translation of declarations**
 - Variable Definition
 - Function Definition

Simple Variable

- For variable v declared in the **current procedure's stack frame**
- Accessing a **local variable v** at offset k (the frame pointer is fp)

MEM (BINOP (PLUS, TEMP fp , CONST k))

- **TEMP fp** is the frame-pointer register
- k is the offset of v within the frame
- For Tiger, all variables are the same size – word size



We can abbreviate **BINOP(PLUS, $e1$, $e2$)** as **+ ($e1$, $e2$)**

Simple Variable, Cont.

- For accessing variable v through static links

```
MEM(+ (CONST  $k_n$ , MEM(+ (CONST  $k_{n-1}$ , ...  
    MEM(+ (CONST  $k_1$ , TEMP FP) ) ...)))
```

- Example: Accessing variable x from inner function (2 levels of nesting):

```
MEM(+ (CONST  $k_x$ ,  
    MEM(+ (CONST  $k_{SL}$ ,  
        MEM(+ (CONST  $k_{SL}$ , TEMP FP) ) ) ) ) )
```

1. $\text{MEM}(+(\text{CONST } k_x, \text{TEMP FP}))$: Access static link in inner frame
2. $\text{MEM}(+(\text{CONST } k_{SL}, \dots))$: Access static link in middle frame
3. $\text{MEM}(+(\text{CONST } k_{SL}, \dots))$: Access variable x in outer frame

Outline of the Translation

- **Translation of expressions**
 - Overview
 - Simple Variables
 - **Array Variables**
 - Structured L-values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- **Translation of declarations**
 - Variable Definition
 - Function Definition

Array and Record Variables in Tiger

- Arrays and records are pointers to sequences of elements/fields
- Assignment copies pointers, not contents
- All variables are scalar (one word)

```
let
  type intArray = array of int
  var a := intArray[12] of 0
  var b := intArray[12] of 7
in a := b /* Now a and b point to same
array */
end
```

- *a* ends up pointing to the same 12 **sevens** as the variable *b*;
- the original 12 **zeros** allocated for *a* are discarded.

NOTE: Arrays are created (and initialized) by $t_a[n] \text{ of } i : t_a$ is the name of an array type, *n* the number of element, *i* the initial value of each element

Array Variables in Other Languages

- In **Pascal**: An array variable stands for the **contents of the array**

```
var a,b : array[1..12] of integer;  
begin  
    a:=b  
end.
```

The snippet **copies the contents of b into a (all 12 ints)**

- In **C**: arrays are like “pointer constants”

```
int a[12], b[12];  
a=b; Illegal
```

**assignment on array
variables are illegal**

```
int a[12], *b;  
b=a; legal
```

**b points to the beginning
of the array a**

Outline of the Translation

- **Translation of expressions**
 - Overview
 - Simple Variables
 - Array Variables
 - **Structured L-values**
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- **Translation of declarations**
 - Variable Definition
 - Function Definition

R-Values and L-Values

- **R-value**: appear on the **right** of an assignment, e.g., $a+3$
 - r-value does not denote an **assignable location**
 - It's a value that can be computed but not assigned to
- **L-value**: the result of an expression that can occur on the left of an assignment statement, e.g., x , $a[i+2]$, $p \rightarrow \text{field}$
 - Denotes a **location** that can be assigned to
 - Can also occur on the right of an assignment statement (it means the contents of the location)

```
x = y + 5;    // y is an l-value used for its contents
              // x is an l-value used as a destination
5 = x;        // Error: 5 is an r-value, not an l-value
```


Scalar vs Structured L-Values

- **Scalar L-value:** An integer or pointer value is a “**scalar**” (It has only one component)
 - All the variables and L-values in Tiger are scalar
 - A Tiger array or record variable is really a pointer
- **Structured L-value:** C or Pascal has **structured L-values**
 - Structs in C, arrays and records in Pascal
 - They are not scalar - have multiple components

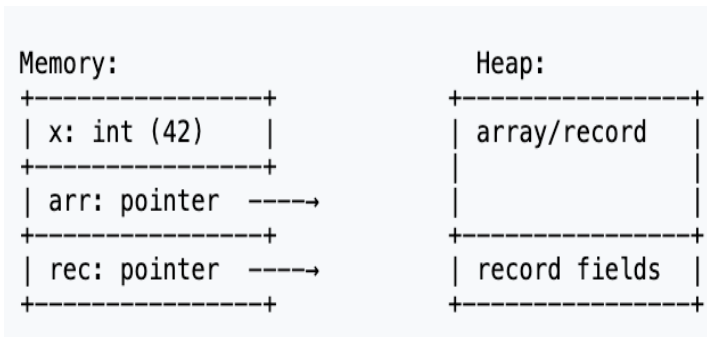
```
struct Point { int x, y; } p1, p2;  
p1 = p2; // Copies entire struct
```

The distinction between scalar and structured L-values significantly impacts how a compiler handles **assignments and memory operations**.

Impact on Memory Representations

Scalar L-values (Tiger)

In Tiger, everything is a scalar :

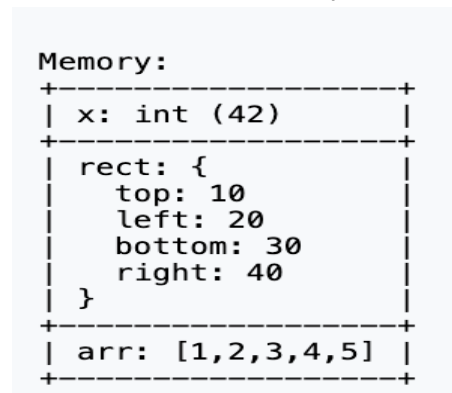


When translating to IR:

- MEM(e) is sufficient to access variables
- All assignments are simple word-sized operations

Structured L-values (Pascal/C)

In Pascal or C, records and arrays can be stored directly:



When translating to IR:

- Need to handle multi-word operations
- Need to track size of values

Tiger avoids complexity by making all variables scalar (one word). Languages that support structured l-values require additional compiler support.

Example: Handling Structured L-Values

- For structured L-values, an address calculation should be $\text{MEM}(+(\text{TEMP fp}, \text{CONST } K), S)$

```
T_exp T_Mem(T_exp, int size);
```

```
Mem(+(TEMP fp,CONST kn), S)
```

- S** indicates the size of the object to be fetched or stored. The size parameter helps the code generator understand:
 - How many bytes need to be loaded/stored
 - What instructions to use (word moves vs block moves)
 - How to handle alignment requirements

Outline of the Translation

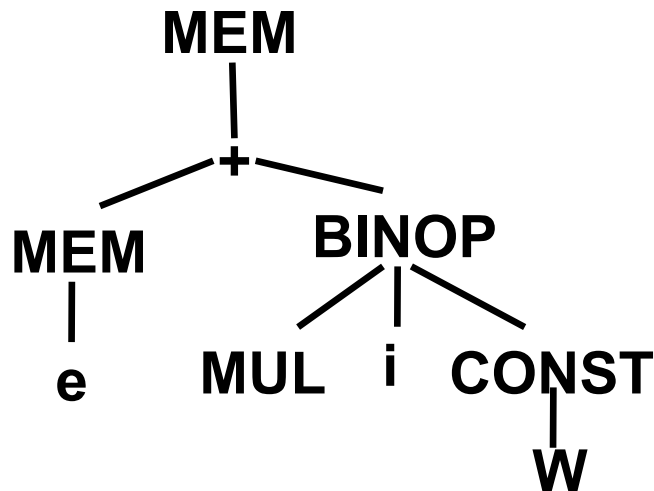
- **Translation of expressions**
 - Overview
 - Simple Variables
 - Array Variables
 - Structured L-values
 - **Subscripting and Field Selection**
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- **Translation of declarations**
 - Variable Definition
 - Function Definition

Subscripting and Field Selection

- In the Tiger language, all record and array values are really **pointers to record and array structures**
 - There is no structural l-value
- The “**base address**” of the array is really the contents of a pointer variable, so **MEM is required**

Subscripting and Field Selection

- To compute the address of $a[i]$:



MEM(BINOP(PLUS,
MEM(e),
BINOP(MUL,
i,
CONST W)))

For an array access $a[i]$ in Tiger

Example: Subscripting and Field Selection

- `arr[i+1]` where `arr` is a local variable at offset 12

```
MEM(BINOP(PLUS,  
    MEM(BINOP(PLUS, TEMP(FP), CONST(12))), /* Load pointer to array */  
    BINOP(MUL,  
        BINOP(PLUS, TEMP(i), CONST(1)), /* Compute i+1 */  
        CONST(4)) /* Multiply by word size */  
))
```

In C notation: `*(*(FP + 12) + (i+1)*4)`

- Similar pattern applies to record field access, but with a constant offset instead of a computed one:

```
// For r.field where field is the n-th field  
MEM(BINOP(PLUS,  
    MEM(e), /* Load pointer to record */  
    CONST(n * W) /* Offset to field */  
)
```

Outline of the Translation

- **Translation of expressions**
 - Overview
 - Simple Variables
 - Array Variables
 - Structured L-values
 - Subscripting and Field Selection
 - **Arithmetic**
 - Conditionals
 - While Loops
 - For Loops
 - Function Call
- **Translation of declarations**
 - Variable Definition
 - Function Definition

Arithmetic

- **Binaries are straightforward**
 - Each arithmetic operator corresponds to a Tree operator (e.g., $\text{BINOP}(o, e1, e2)$)
- **No unary operator!**
 - Unary negation can be implemented as subtraction from zero
 - Unary complement can be implemented as XOR with all ones
- No floating point

Outline of the Translation

- **Translation of expressions**
 - Overview
 - Simple Variables
 - Array Variables
 - Structured L-values
 - Subscripting and Field Selection
 - Arithmetic
 - **Conditionals**
 - While Loops
 - For Loops
 - Function Call
- **Translation of declarations**
 - Variable Definition
 - Function Definition

Conditionals (重点)

- The result of a comparison operator will be a **Cx** expression (A statement **s** that will jump to any true-destination and false-destination)

```
if x < 5 then a := 1
else a := 2
```

```
SEQ(CJUMP(LT, x, CONST 5, t, f),
    SEQ(LABEL t,
        SEQ(MOVE(TEMP a, CONST 1),
            SEQ(JUMP(NAME join, [join]),
                SEQ(LABEL f,
                    SEQ(MOVE(TEMP a, CONST 2),
                        LABEL join))))))
```

Conditionals: About Short-Circuit

- Tiger的逻辑运算符 $\&$ (and)和 $|$ (or)需要实现短路求值：
只计算必要的操作数

逻辑与 ($a \& b$)的短路规则：

1. 计算a
2. 若a为假，直接得到假结果（不计算b）
3. 若a为真，继续计算b，最终结果即为b的值

逻辑或 ($a | b$)的短路规则：

1. 计算a
2. 若a为真，直接得到真结果（不计算b）
3. 若a为假，继续计算b，最终结果即为b的值

Example: Short-Circuit for AND

/ 简单的逻辑与表达式示例 */*

```
if a > 0 & b/a > 5 then
    print("Valid")
else
    print("Invalid")
```

1. 先判断 $a > 0$: 若为假; 跳转到 label_false (绕过第二个条件的计算); 若为真, 继续到 label_continue
2. 在 label_continue 处, 计算并判断 b/a : 若为真, 跳转到 label_true; 若为假, 跳转到 label_false

// 逻辑与表达式的IR树

```
SEQ(
  CJUMP(GT, TEMP a, CONST 0, label_continue, label_false),
  LABEL label_continue,
  CJUMP(GT, DIV(TEMP b, TEMP a), CONST 5, label_true, label_false),
  LABEL label_true,
    EXP(CALL(NAME print, CONST "Valid")),
    JUMP(NAME label_end),
  LABEL label_false,
    EXP(CALL(NAME print, CONST "Invalid")),
  LABEL label_end
)
```

Example: Short-Circuit for OR

```
/* 简单的逻辑或表达式示例 */  
if a < 0 | a > 100 then  
    print("Out of range")  
else  
    print("In range")
```

- 1.第一个条件判断: $a < 0$, 若为真则直接跳转到label_true
- 2.只有当第一个条件为假时, 才会继续到label_next计算第二个条件
- 3.第二个条件判断: $a > 100$, 根据结果跳转到label_true或label_false

// 逻辑或表达式的IR树

```
SEQ(  
    CJUMP(LT, TEMP a, CONST 0, label_true, label_next),  
    LABEL label_next,  
    CJUMP(GT, TEMP a, CONST 100, label_true, label_false),  
    LABEL label_true,  
        EXP(CALL(NAME print, CONST "Out of range")),  
        JUMP(NAME label_end),  
    LABEL label_false,  
        EXP(CALL(NAME print, CONST "In range")),  
    LABEL label_end  
)
```

Outline of the Translation

- **Translation of expressions**
 - Overview
 - Simple Variables
 - Array Variables
 - Structured L-values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - **While Loops**
 - For Loops
 - Function Call
- Translation of declarations
 - Variable Definition
 - Function Definition

While Loops

- The general layout of a **while** loop is:

```
test:
    if not(condition) goto done
    body
    goto test
done:
```

```
while a > 0 do
    a := a - 1
```



```
SEQ(LABEL test,
    SEQ(CJUMP(GT, TEMP a, CONST 0, body, done),
        SEQ(LABEL body,
            SEQ(MOVE(TEMP a, BINOP(MINUS, TEMP a, CONST 1)),
                SEQ(JUMP(NAME(test), [test]),
                    LABEL done))))))
```


While Loops: Handling Break

```
while a > 0 do  
  (if a = 5 then break;  
   a := a - 1)
```



break翻译为直接跳转到done

```
SEQ(LABEL test,  
  SEQ(CJUMP(GT, TEMP a, CONST 0, body, done),  
    SEQ(LABEL body,  
      SEQ(CJUMP(EQ, TEMP a, CONST 5, t, f),  
        SEQ(LABEL t,  
          SEQ(JUMP(NAME done, [done])),  
          SEQ(LABEL f,  
            SEQ(MOVE(TEMP a, BINOP(MINUS, TEMP  
a, CONST 1))),  
            SEQ(JUMP(NAME test, [test]),  
              LABEL done))))))))))
```

Outline of the Translation

- **Translation of expressions**
 - Overview
 - Simple Variables
 - Array Variables
 - Structured L-values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - **For Loops**
 - Function Call
- **Translation of declarations**
 - Variable Definition
 - Function Definition

For Loops

- A straightforward approach to translate **for** statements
 - Rewrite the *abstract syntax* into the abstract syntax of the **let/while** expression shown.

```
for i := lo to hi  
do body
```



```
let  
  var i := lo  
  var limit := hi  
in  
  while i <= limit  
  do (body; i := i+1)  
end
```

For Loops

- A straightforward approach to translate **for** statements

```
for i := lo to hi  
do body
```



```
let var i := lo  
    var limit := hi  
in while i <= limit  
    do (body; i := i+1)  
end
```

- Problem:** When *limit*=*maxint*, *i+1* will overflow!
- 将测试放在循环底部：在递增*i*之前检查*i* < *limit*，而不是检查*i* <= *limit*:样可以避免在*i*已经达到*maxint*时执行*i+1*
- 添加初始范围检查：在进入循环前先检查*lo* <= *hi*: 如果初始条件不满足，可以直接跳过整个循环

```
if lo > hi goto done  
i := lo  
limit := hi  
test:  
    body  
    if i >= limit goto done  
    i := i+1  
    goto test  
done:
```

Outline of the Translation

- **Translation of expressions**
 - Overview
 - Simple Variables
 - Array Variables
 - Structured L-values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - **Function Calls**
- **Translation of declarations**
 - Variable Definition
 - Function Definition

Function Call

- Translating a function call $f(a_1, \dots, a_n)$ is simple, except that the **static link** must be added as an implicit extra argument:

$\text{CALL}(\text{NAME } l_f, [sl, e_1, e_2, \dots, e_n])$

- l_f is the label for f ,
- sl is the static link --- a pointer to f 's **parent level**

Both the level of f and the level of the function calling f are required to calculate sl

Outline of the Translation

- **Translation of expressions**
 - Overview
 - Simple Variables
 - Array Variables
 - Structured L-values
 - Subscripting and Field Selection
 - Arithmetic
 - Conditionals
 - While Loops
 - For Loops
 - Function Calls
- **Translation of declarations**
 - Variable Definition
 - Function Definition

Declarations

- **Variable declaration**

Need to figure out the **offset** in the frame, then **move** the expression on the r.h.s. to the proper **slot** in the **frame**.

- **Type declaration**

No need to generate any IR tree code !

- **Function declaration**

```
name:      _global name
           .....
           assembly code for body
           .....
                                     prologue
                                     epilogue
```


Variable Declarations

- Reserve space in current frame
- Generate initialization expressions
- Execute initializations before body of let

```
let
  var x := 10
  var y := x + 5
in
  x + y
end
```

```
ESEQ(
  SEQ(
    MOVE(MEM(+ (FP, CONST(x_offset))), CONST(10)),
    MOVE(MEM(+ (FP, CONST(y_offset))),
      BINOP(PLUS,
        MEM(+ (FP, CONST(x_offset))),
        CONST(5)))
  ),
  BINOP(PLUS,
    MEM(+ (FP, CONST(x_offset))),
    MEM(+ (FP, CONST(y_offset))))
)
```

Function Declarations

- Function is translated into “assembly language segment” with three components
 - **Prologue**
 - **Body**
 - **Epilogue**

```
name:      _global name
          ..... prologue
          assembly code for body
          ..... epilogue
```

Function Declarations: Prologue

- **Generating the **prologue**:**
 1. Psuedo-instructions: announce function start
 2. Label definiton fo the function name
 3. Adjust the stack pointer (allocating frame)
 - 4.Store** instructions to save callee-save registers and return address
 - 5.Store** instructions to save arguments and static links

Items 1, 3 depend on **final frame size** →
generated **late** (after register allocation)

Function Declarations: Body and Epilogue

- **Generating the **body**:** a Tiger function is an expression
 6. The translated expression
- **Generating the **epilogue**:**
 7. Move the return result to a special register
 - 8. Load** instructions to restore callee-save registers
 9. Reset the stack pointer (pop the frame)
 - 10. Return** instruction (jump to the return address)
 11. Psuedo-instructions to announce the end of a function

Function Declarations

- Steps 1, 3, 9, 11 depend on exact size of stack frame.
- These are generated late (after register allocation).

1. Pseudo-instructions to **mark the beginning of a function** (needed in the particular assembly language)
2. A **label definition** of the function name
3. An instruction to **adjust the stack pointer** (to allocate a new frame)
4. Instructions to **save “escaping” arguments** (including the static link) into the frame, and to **move nonescaping arguments** into fresh temporary registers
5. Store instructions to **save any callee-save registers** (including the return address register) used within the function
6. The translated expression for body
7. An instruction to **move the return value** (result of the function) to the register
8. Load instructions to **restore the callee-save registers**
9. An instruction to **reset the stack pointer** (to *deallocate the frame*)
10. A **return** instruction (**JUMP** to the return address)
11. Pseudo-instructions, as needed, to **announce the end of a function**

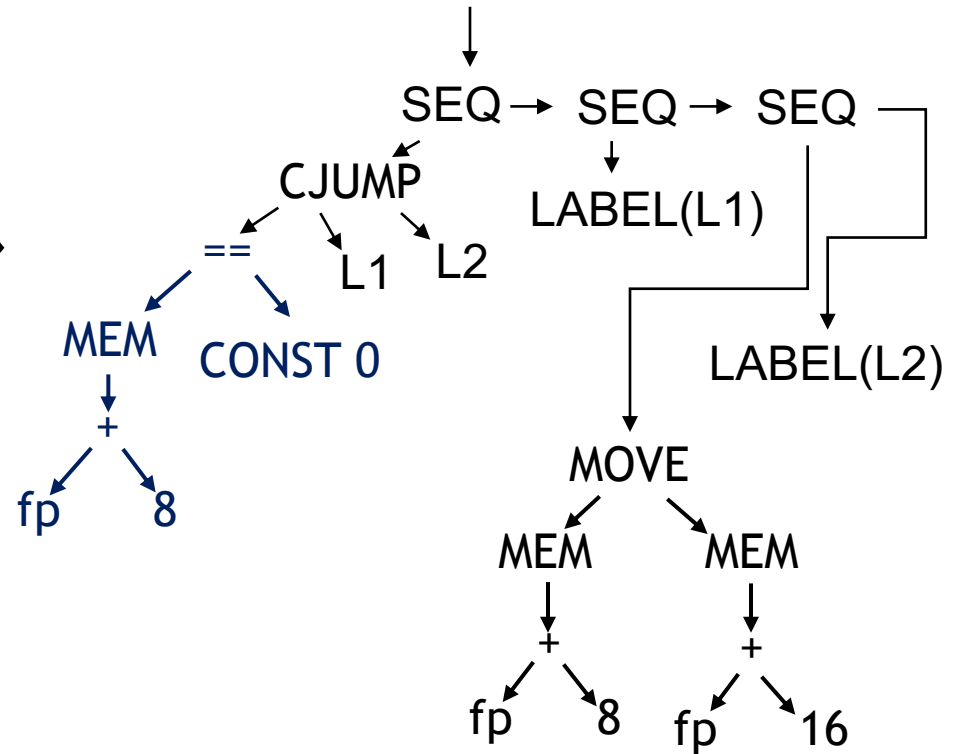
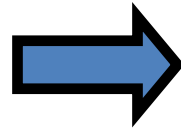
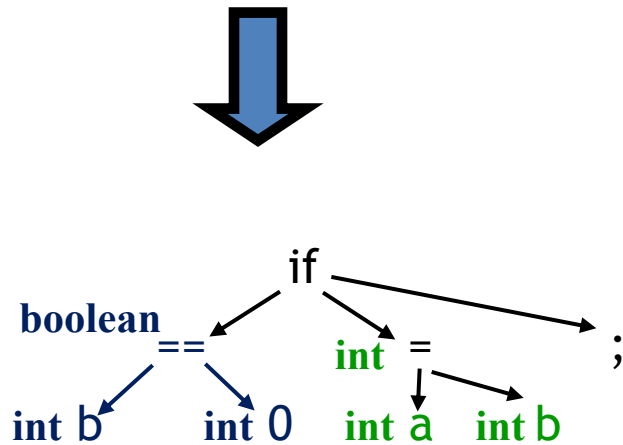
Key Design Decisions Summary

Decision	Rationale
Simple expression trees as IR	Each node = one simple operation
MEM has dual meaning	Handles l-values and r-values uniformly
Three kinds: Ex, Nx, Cx	Efficient translation of different expression types
Patch lists for Cx	Short-circuit booleans without knowing labels yet
Infinite temporaries	Defers register allocation to later phase
No function defs in IR	Prologue/epilogue are machine-specific
Fragments as output	Clean separation of string data and code



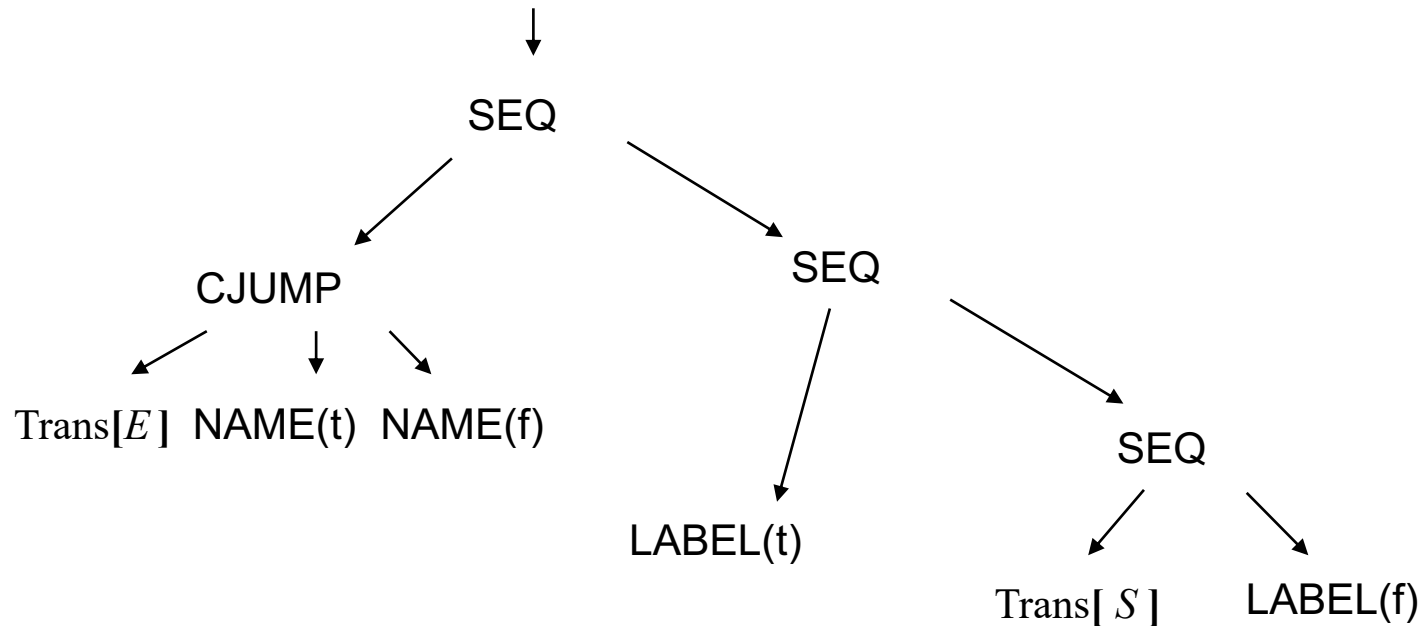
Example: Tiger AST to IR Tree

if (b==0) a = b;



Example: Tiger AST to IR Tree

• $\text{Trans} [\text{if} (E) S] = ?$



= `SEQ(CJUMP(Trans[E], NAME(t), NAME(f)),
SEQ(LABEL(t),
SEQ(Trans[S], LABEL(f)));`